

Lecture 6: Kernels (and Triton)

Kernel Programming

Recall, a kernel is a function, that executes on a GPU!

- *Thread*: executes code on a small part of the data.
- *Thread block or concurrent thread array (CTA)*: a group of threads.
- *Grid*: collection of thread blocks.

(H100s and B200s also have thread block clusters that enable distributed shared memory.)

Question: *Why thread blocks?*

For elementwise operations (e.g., GeLU), threads are most natural: each thread processes one element.

- $f(i)$ for $i = 0, \dots, N - 1$.

However, for non-elementwise operations like softmax or matrix multiplication, threads need to communicate.

- Reading/writing from HBM is slow, so use shared memory (local to SM).
- A block of threads access the same shared memory.
- Consequently, a thread block is scheduled on one SM.
- In Triton, think natively in terms of thread blocks (later).

Hardware Specification

Accelerator	A100	H100	B200
# SMs	108	132	148
Register size	256 KB	256 KB	256 KB
L1 cache + shared memory	192 KB	256 KB	256 KB
L2 cache size	40 MB	50 MB	96–126 MB
HBM size	80 GB	80 GB	192 GB
Register bandwidth	~116 TB/s	~401 TB/s	~447 TB/s
L1 cache + shared bandwidth	~19 TB/s	~33 TB/s	~40 TB/s
L2 cache bandwidth	~5–8 TB/s	~12 TB/s	~17 TB/s
HBM bandwidth	2 TB/s	3.35 TB/s	8 TB/s

Note: the L1/L2 bandwidth numbers are rough estimates (online sources differ).

(B200s also have tensor memory (TMEM) for tensor cores (between registers and shared memory) that are invisible to programmer.)

Interaction between Software and Hardware

Programming model provides an abstraction of the hardware.

- In principle, don't need to think about anything else (for correctness).
- In practice, performance is very sensitive to the hardware, so need to understand it to obtain high performance.

Let's go over some considerations.

Warp Execution

- Within a thread block, threads are grouped into warps (32 threads per warp).
- Example: thread block has 64 threads $\implies$ it has 2 warps.
- All threads within a warp must execute same instructions in lockstep on an SM.
- Control divergence: if different threads in a warp need to execute different instructions (if A, else B), must be done sequentially (bad).
- SM runs multiple warps and switches between them (e.g., when one warp is blocked on HBM reads/writes) with zero cost.

(Warp) Occupancy

- Each thread can use between 0 and 255 registers.
- The more registers threads use, the fewer threads can be scheduled on an SM.
- Low occupancy isn't necessarily bad if each thread is doing more work.
- Example: thread coarsening (each thread processes multiple elements).
- Example: thread block has 64 threads, each using 160 registers, SM has 65536 registers.

```
# What we want to run
threads_per_block = 128
registers_per_thread = 160
```

```
# What hardware offers
max_registers = 65536 # Registers allowed per SM
max_warps = 64 # Concurrent warps allowed per SM

# What we can run at once
assert registers_per_thread <= 255
registers_per_block = threads_per_block * registers_per_thread
num_blocks = max_registers // registers_per_block

# Limited by registers
warps = blocks * threads_per_block / 32
occupancy = warps / max_warps
```

Bank Conflicts (shared memory)

- Shared memory is divided into 32 banks, each 4 bytes wide.
- Each cycle, each bank can only be accessed by one thread (if not the same exact location).
- If multiple threads access the same bank, accesses serialized (bank conflict).
- Worst case example: matrix where each row spans all banks; 32 threads accessing first column results in 32-way bank conflict!
- Unavoidable: when doing matmul $A @ B$, access rows of A and columns of B .
- Solution: swizzling rearranges shared memory (e.g., row xor col) to avoid bank conflicts.

Memory Coalescing (HBM)

- When the 32 threads in a warp access HBM, memory accesses combined into transactions of 128 bytes (cache lines).
- Best case: full coalescing, all threads access the same cache line (32 threads $\times$ 4 bytes = 128 bytes).

Block Occupancy

- Thread blocks scheduled onto SMs in waves.
- B200 has 148 SMs, if we launch 160 thread blocks, first wave has 148 blocks, second wave has 12 blocks.
- Wave quantization problem: last wave has fewer thread blocks, leaving some SMs idle (low block occupancy).
- Solution: make number of thread blocks divide # SMs.

Summary

- Programming model: grid (HBM) → thread block (shared memory) → thread (registers).
- GPU architecture: details (warps, bank conflicts, memory coalescing, occupancy) matter in performance.

Benchmarking and Profiling

Recipe for success:

1. Benchmark and profile your code!
2. Make changes!
3. Benchmark and profile your code again!

Benchmarking

Benchmarking measures the wall-clock time of performing some operation. It only gives you end-to-end time, not where time is spent (profiling).

```
def run_operation1(dim: int, operation: Callable) -> Callable:
    # Setup: create one random dim x dim matrices
    x = torch.randn(dim, dim, device=cuda_if_available())
    # Return a function to perform the operation
    return lambda : operation(x)
```

It is still useful for:

- comparing different implementations (which is faster?) and
- understanding how performance scales (e.g., with dimension).

```
def run_operation2(dim: int, operation: Callable) -> Callable:
    # Setup: create two random dim x dim matrices
    x = torch.randn(dim, dim, device=cuda_if_available())
    y = torch.randn(dim, dim, device=cuda_if_available())

    # Return a function to perform the operation
    return lambda : operation(x, y)
```

You can use **torch.util.benchmark**. We will roll our own to make benchmarking more transparent.

```
def benchmark(run: Callable,
              num_warmups: int = 1, num_trials: int = 3) -> float:
    # Warmup: first times might be slower due to compilation.
    for _ in range(num_warmups):
        run()
    torch.cuda.synchronize()

    # Time it for real now!
    times: list[float] = []

    # Do it multiple times to capture variance
    for trial in range(num_trials):
        # Use CUDA events for accurate GPU timing
        start_event = torch.cuda.Event(enable_timing=True)
        end_event = torch.cuda.Event(enable_timing=True)
        start_event.record() # Start timing
        run() # Actually perform computation
        end_event.record() # End timing
        torch.cuda.synchronize()
        times.append((start_event.elapsed_time(end_event)))
    mean_time = mean(times)
    return mean_time
```

Note: time is roughly constant when dimension is small, then cubic scaling.

```
# Benchmark matrix multiplication
matmul = run_operation2(dim=1024, operation=lambda a, b: a @ b)
result = benchmark(matmul)

# See how timing scales with dimension
results = {}
for dim in [256, 512, 1024, 2048, 4096, 8192]:
    results[dim] = benchmark(
        run_operation2(dim=dim, operation=lambda a, b: a @ b)
    )
```

Profiling

While benchmarking looks at end-to-end time, profiling looks at where time is spent.

- Independent of time, profiling also helps you understand what's going under the hood.
- PyTorch has a built-in profiler. In your assignment, you will use nsight to get more details.

Observations:

- You can see which CUDA kernels are actually being called (the long names).
- Different CUDA kernels are invoked depending on the tensor dimensions.

Name of CUDA kernel tells us something about the implementation.

- **kernel**: cutlass3x_sm100_simt_sgemm_f32_f32_f32_f32_f32_64x64x16...
- **cutlass**: NVIDIA's CUDA library for linear algebra.
- **sm100**: corresponds to the NVIDIA Blackwell architecture (B200).
- **f32**: single-precision float-pointing representation.
- **64x64x16**: tile shape (more on this later).

```
def profile(run: Callable, num_warmups: int = 1):
    # Warmup
    for _ in range(num_warmups):
        run()
    torch.cuda.synchronize()

    # Run the code with the profiler
    with torch.profiler.profile(
        activities=[ProfilerActivity.CUDA], ...) as prof:
        run()
        torch.cuda.synchronize()

    # Print out table
    table = prof.key_averages().table(
        sort_by="cuda_time_total",
        max_name_column_width=100,
        row_limit=10)

    # Append to profiles.txt
    with open("var/profiles.txt", "a") as f:
        f.write(f"Profile at {time.ctime()}: \n")
        f.write(table)
        f.write("\n\n")
```

Naive vs Builtin vs Compiled GeLU

```
def naive_gelu(x: torch.Tensor):
    # tanh approximation to the gelu activation function
    return 0.5 * x * (
        1 + torch.tanh(0.79788456 * (x + 0.044715 * x * x * x)))
```

Let's benchmark and profile the GeLU activation function.

```
x = torch.tensor([1.])

# 1. Implementation naively from scratch in PyTorch (non-fused)
y1 = naive_gelu(x)

# 2. Built-in PyTorch implementation (fused)
y2 = builtin_gelu(x)

# Check it works
check_equal_1d(naive_gelu, builtin_gelu)

# 3. Use PyTorch compiler on the naive implementation
compiled_gelu = torch.compile(naive_gelu)
y3 = compiled_gelu(x)

# Check it works (compilation shouldn't change semantics)
check_equal_1d(naive_gelu, compiled_gelu)

# Benchmarking
naive_time = benchmark(
    run_operation1(dim=16384, operation=naive_gelu))
builtin_time = benchmark(
    run_operation1(dim=16384, operation=builtin_gelu))
compiled_time = benchmark(
    run_operation1(dim=16384, operation=compiled_gelu))
```

The builtin and compiled versions are significantly faster! To understand why, let's look at the profiler to see where time is being spent.

```
def builtin_gelu(x: torch.Tensor):
    # PyTorch's built-in GeLU with the tanh approximation
    return torch.nn.functional.gelu(x, approximate="tanh")
```

Notes:

- *Naive implementation*: multiple kernels, requires many reads/writes from/to HBM.
- *Builtin and compiled versions*: one fused kernel, one read from HBM, one write to HBM.
- The compiled kernel is a Triton kernel.

```
def check_equal_1d(f1, f2):
    x = torch.randn(2048, device=cuda_if_available())
    y1 = f1(x)
    y2 = f2(x)
    assert torch.allclose(y1, y2, atol=1e-6)
```

Triton Introduction

In CUDA (developed by NVIDIA), specify what each thread does.

- Pros: fine-grained control.
- Cons: need to manage more things (e.g., shared memory).

In Triton (developed by OpenAI), specify what each thread block does.

- Generally powerful enough (especially when getting started).
- Conceptual framework: load data into shared memory, operate on it, write back to global memory.

GeLU Example

Let's write the Triton kernel for GeLU.

```
x = torch.randn(8192, device=cuda_if_available())
y = triton_gelu(x)

# Check for correctness
check_equal_1d(triton_gelu, naive_gelu)
```

Triton compiles down to PTX (parallel thread execution), an assembly language for GPUs:

- `ld.global.*` and `st.global.*` reads and writes from global memory.
- `%ctaid.x` is block index, `%tid.x` is thread index.
- `%f*` are floating point registers, `%r*` are integer registers.
- One thread processes 8 elements at the same time (thread coarsening).

Compiled GeLU

```
def triton_gelu(x: torch.Tensor):
    # Check input
    assert x.is_cuda
    assert x.is_contiguous()

    # Allocate output tensor
    y = torch.empty_like(x)

    # Determine grid (elements divided into blocks)
    # | T T T T T T T T | T T T T T T T T | T T T T T T T T |
    # |      Block 0      |      Block 1      |      Block 2      |
    num_elements = x.numel()
    BLOCK_SIZE = 1024 # Number of threads
    num_blocks = triton.cdiv(num_elements, BLOCK_SIZE)

    # Launch the kernel
    kernel = triton_gelu_kernel[(num_blocks,)](
        x, y, num_elements, BLOCK_SIZE=BLOCK_SIZE)

    # Write out PTX (look at this later)
    output_ptx("triton_gelu", kernel)
```

GeLU Kernel

```
def triton_gelu_kernel(x_ptr, y_ptr,
                       num_elements, BLOCK_SIZE: tl.constexpr):
    # Input starts at `x_ptr`
    # Output starts at `y_ptr`
    # | T T T T T T T T | T T T T T T T T | T T T T T T T T |
    # |      Block 0      |      Block 1      |      Block 2      |
    pid = tl.program_id(axis=0)
    start = pid * BLOCK_SIZE

    # Indices where this thread block should operate
    offsets = start + tl.arange(0, BLOCK_SIZE)

    # Don't read/write past the end of the tensor
    mask = offsets < num_elements

    # Read the bytes
    x = tl.load(x_ptr + offsets, mask=mask)
```

```
# Compute the GeLU (tl.tanh doesn't exist)
a = 0.79788456 * (x + 0.044715 * x * x * x)
exp = tl.exp(2 * a)
tanh = (exp - 1) / (exp + 1)
y = 0.5 * x * (1 + tanh)

# Store the bytes
tl.store(y_ptr + offsets, y, mask=mask)
```

Triton Softmax Example

So far, we've looked at elementwise operations in Triton (e.g., GeLU). Now let us look at operations that aggregate over multiple values.

```
def check_equal_2d(f1, f2):
    x = torch.randn(2048, 2048, device=cuda_if_available())
    y1 = f1(x)
    y2 = f2(x)
    assert torch.allclose(y1, y2, atol=1e-6)
```

Recall the softmax operation is used in attention and generating probabilities. Exponentiate and normalize each row of a matrix:

$$\begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & -\infty \end{bmatrix} \Rightarrow \begin{bmatrix} 1/3 & 1/3 & 1/3 \\ 1/2 & 1/2 & 0 \end{bmatrix}$$

Let's first start with the naive implementation and keep track of reads/writes.

```
x = torch.tensor([
    [5., 5, 5],
    [0, 0, 100],
], device=cuda_if_available())
y1 = naive_softmax(x)
```

Now let us write the Triton kernel.

```
y2 = triton_softmax(x)

# Check our implementations are correct
check_equal_2d(pytorch_softmax, naive_softmax)
check_equal_2d(pytorch_softmax, triton_softmax)
```

Naive Softmax

```
def naive_softmax(x: torch.Tensor):
    # M: number of rows, N: number of columns
    M, N = x.shape

    # Compute the max of each row (MN reads, M writes)
    x_max = x.max(dim=1)[0]

    # Subtract off the max (MN + M reads, MN writes)
    x = x - x_max[:, None]

    # Exponentiate (MN reads, MN writes)
    numerator = torch.exp(x)

    # Compute normalization constant (MN reads, M writes)
    denominator = numerator.sum(dim=1)

    # Normalize (MN reads, MN writes)
    y = numerator / denominator[:, None]

    # Total: 5MN + M reads, 3MN + 2M writes
    # In principle, should have MN reads, MN writes (speedup of 4x!)
    return y
```

Compiled Softmax

```
def triton_softmax(x: torch.Tensor):
    # Allocate output tensor
    y = torch.empty_like(x)

    # Determine grid
    M, N = x.shape
    block_size = triton.next_power_of_2(N)
    num_blocks = M

    # Launch kernel
    triton_softmax_kernel[(M,)](
        x_ptr=x, y_ptr=y,
        x_row_stride=x.stride(0), y_row_stride=y.stride(0),
        num_cols=N, BLOCK_SIZE=block_size
    )
```

Softmax Kernel

```
def triton_softmax_kernel(x_ptr, y_ptr,
                          x_row_stride, y_row_stride,
                          num_cols, BLOCK_SIZE: tl.constexpr):
    assert num_cols <= BLOCK_SIZE
    # Process each row independently
    row_idx = tl.program_id(0)
    col_offsets = tl.arange(0, BLOCK_SIZE)

    # Read from global memory
    x_start_ptr = x_ptr + row_idx * x_row_stride
    x_ptrs = x_start_ptr + col_offsets
    x_row = tl.load(
        x_ptrs, mask=col_offsets < num_cols, other=float("-inf"))

    # Compute
    x_row = x_row - tl.max(x_row, axis=0)
    numerator = tl.exp(x_row)
    denominator = tl.sum(numerator, axis=0)
    y_row = numerator / denominator

    # Write back to global memory
    y_start_ptr = y_ptr + row_idx * y_row_stride
    y_ptrs = y_start_ptr + col_offsets
    tl.store(y_ptrs, y_row, mask=col_offsets < num_cols)
```

Triton Rowsum Example

In the softmax example, an entire row fits in a block, so the reduction happens within a block (handled by Triton). What if the row doesn't fit in a block?

Example: 4096 columns, but block size is 1024...

Strategy:

- Break up row into tiles (4 in the example above).
- Each thread iterates over tiles and accumulates a sum.
- Do final reduction (sum) over accumulators of each thread (shared memory or warp shuffles).

Consider the simpler example (row sum instead of softmax):

```
x = torch.tensor([
    [1., 2, 3, 4], [5, 6, 7, 8]], device=cuda_if_available())
y1 = builtin_row_sum(x)
y2 = triton_row_sum(x)
```

Builtin Rowsum

```
def builtin_row_sum(x: torch.Tensor):
    return x.sum(dim=1)
```

Compiled Rowsum

```
def triton_row_sum(x: torch.Tensor,
                   BLOCK_SIZE: int = 1024) -> torch.Tensor:
    M, N = x.shape
    y = torch.empty(M, device=x.device, dtype=x.dtype)
    row_sum_kernel[(M,)](x, y, N, BLOCK_SIZE=BLOCK_SIZE)
    return y
```

Rowsum Kernel

```
@triton.jit
def row_sum_kernel(x_ptr, out_ptr, N, BLOCK_SIZE: tl.constexpr):
    row = tl.program_id(0) # Which row are we processing?

    # Accumulator for each thread
    # One row: T1 T2 T3 T4 | T1 T2 T3 T4 | T1 T2 T3 T4
    acc = tl.zeros([BLOCK_SIZE], dtype=tl.float32)

    # Loop over tiles
    for start in range(0, N, BLOCK_SIZE):
        cols = start + tl.arange(0, BLOCK_SIZE)
        mask = cols < N
        x = tl.load(x_ptr + row * N + cols, mask=mask, other=0.0)
        acc += x

    # Final reduction from BLOCK_SIZE (all threads) to a scalar
    result = tl.sum(acc, axis=0)
    tl.store(out_ptr + row, result)
```

Matrix Multiplication

Matrix multiplication is the bread and butter of deep learning.

```

a = torch.randn(1024, 1024, device=cuda_if_available())
b = torch.randn(1024, 1024, device=cuda_if_available())
c = naive_matmul_relu(a, b)

```

How should we build a matmul kernel?

$$\begin{array}{c} \text{m} \end{array} \begin{array}{c} \text{k} \\ \begin{bmatrix} A1 & A2 & A3 \\ A4 & A5 & A6 \\ A7 & A8 & A9 \end{bmatrix} \end{array} * \begin{array}{c} \text{k} \\ \begin{bmatrix} B1 & B2 & B3 \\ B4 & B5 & B6 \\ B7 & B8 & B9 \end{bmatrix} \end{array} \begin{array}{c} \text{n} \\ \begin{bmatrix} C1 & C2 & C3 \\ C4 & C5 & C6 \\ C7 & C8 & C9 \end{bmatrix} \end{array}$$

Naive approach:

- Read $A[m, k]$ and $B[k, n]$ from HBM.
- Multiply and accumulate.
- Write result to $C[m, n]$ in HBM.

Bottleneck: MKN reads, MN writes and $O(1)$ intensity.

- Computing $C4$ and $C5$ both need $A4, A5, A6$.
- Can we read $A4, A5, A6$ from HBM once to compute both?
- Yes, using shared memory!

Idealized approach:

- Load all of A and B into shared memory, then compute C .
- Now we get $MK + KN$ reads and MN writes.
- This yields the idealized $O(N)$ arithmetic intensity from before.
- However, A and B are usually too large to fit in shared memory.

Tiling approach:

- Key idea: divide the matrix C into output tiles (thread blocks).
- Fix an output tile in C .

For each pair of (row tile of A , column tile of B):

- Load the corresponding A tile and B tile from HBM into shared memory.
- Perform matrix multiplication on the tiles.

- Accumulate into the partial sum (in shared memory).

Write output tile to HBM.

Arithmetic intensity: $O(\text{tile_size})$.

Bonus content:

- Often, you want to apply an elementwise activation function.
- Example: $\text{GeLU}(AB)$.
- Solution: kernel fusion!
- Review: each matrix is linearized in memory.

```
x = torch.tensor([[0., 1, 2, 3], [4, 5, 6, 7]])
stride_row, stride_col = x.stride()
row = 1
col = 2
index = row * stride_row + col * stride_col

# Compute c = a
c = triton_matmul_relu(a, b)
```

Naive Matmul ReLU

```
def naive_matmul_relu(x: torch.Tensor, y: torch.Tensor):
    # Matmul followed by ReLU
    return torch.nn.functional.relu(x @ y)
```

Compiled Matmul ReLU

```
def triton_matmul_relu(a: torch.Tensor, b: torch.Tensor):
    assert a.is_cuda and b.is_cuda
    assert a.is_contiguous() and b.is_contiguous()
    assert a.shape[1] == b.shape[0]

    # A is M x K, B is K x N
    M, K = a.shape
    K, N = b.shape

    # Allocate output tensor
    c = torch.empty((M, N), device=a.device)
```

```

# Determine grid
BLOCK_M, BLOCK_N, BLOCK_K = 64, 64, 32
grid = (triton.cdiv(M, BLOCK_M), triton.cdiv(N, BLOCK_N))
matmul_relu_kernel(grid)(
    a, b, c,
    M, N, K,
    a.stride(0), a.stride(1),
    b.stride(0), b.stride(1),
    c.stride(0), c.stride(1),
    BLOCK_M, BLOCK_N, BLOCK_K,
)
return c

```

Matmul ReLU Kernel

```

def matmul_relu_kernel(
    a_ptr, b_ptr, c_ptr,      # Compute c = a
    M, N, K,                  # a is M x K, b is K x N, c is M x N
    stride_am, stride_ak,     # How to navigate a
    stride_bk, stride_bn,     # How to navigate b
    stride_cm, stride_cn,     # How to navigate c
    BLOCK_M: tl.constexpr,
    BLOCK_N: tl.constexpr,
    BLOCK_K: tl.constexpr,
):
    # We are working on the (m, n)-th tile
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)

    # Indices
    indices_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    indices_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
    indices_k = tl.arange(0, BLOCK_K)

    # Initial matrix of pointers of a and b
    a_ptrs = a_ptr +
        indices_m[:, None] * stride_am +
        indices_k[None, :] * stride_ak
    b_ptrs = b_ptr +
        indices_k[:, None] * stride_bk +
        indices_n[None, :] * stride_bn
    acc = tl.zeros([BLOCK_M, BLOCK_N], dtype=tl.float32)

```



```
# Move along row tiles of a, column tiles of b
for k in range(0, K, BLOCK_K):
    a = tl.load(a_ptrs,
                mask=(indices_m[:, None] < M) &
                    (indices_k[None, :] + k < K), other=0.0)
    b = tl.load(b_ptrs,
                mask=(indices_k[:, None] + k < K) &
                    (indices_n[None, :] < N), other=0.0)
    acc += tl.dot(a, b)
    a_ptrs += BLOCK_K * stride_ak
    b_ptrs += BLOCK_K * stride_bk

# Apply activation function (e.g., ReLU)
acc = tl.maximum(acc, 0.0)

# Write output tile
c_ptrs = c_ptr +
    indices_m[:, None] * stride_cm +
    indices_n[None, :] * stride_cn
tl.store(c_ptrs,
        acc, mask=(indices_m[:, None] < M) &
                (indices_n[None, :] < N))
```